The root entity of this Domain Model is the `Board` entity because, either directly or indirectly, all the other entities are associated with a `Board`.

```
@Entity @Table(name = "board")
public class Board {

    @Id @GeneratedValue
    private Long id;

    private String name;

    //Getters and setters omitted for brevity
}
```

The end user can submit either a `Post` or an `Announcement` on a particular `Board`. Because the `Post` and the `Announcement` share the same functionality (differing only in data), they both inherit from a `Topic` base class.

The `Topic` class defines a relationship to a `Board` entity, hence the `Post` and the `Announcement` entities can also be associated with a `Board` instance.

```
@Entity @Table(name = "topic")
public class Topic {

    @Id @GeneratedValue
    private Long id;

    private String title;

    private String owner;

    @Temporal(TemporalType.TIMESTAMP)
    private Date createdOn = new Date();

    @ManyToOne(fetch = FetchType.LAZY)
    private Board board;

    //Getters and setters omitted for brevity
}
```

Both the `Post` and the `Announcement` entities extend the `Topic` class and define their own specific attributes.

```
@Entity @Table(name = "post")
public class Post extends Topic {

    private String content;

    //Getters and setters omitted for brevity
}

@Entity @Table(name = "announcement")
public class Announcement extends Topic {

    @Temporal(TemporalType.TIMESTAMP)
    private Date validUntil;

    //Getters and setters omitted for brevity
}
```

The `TopicStatistics` is at the bottom of this Domain Model as it is only needed for monitoring purposes, without being directly associated with the main business logic. Because statistics are needed for both `Post` and `Announcement` entities, the `TopicStatistics` defines a `Topic` entity association.

```
@Entity @Table(name = "topic_statistics")
public class TopicStatistics {

    @Id @GeneratedValue
    private Long id;

    @OneToOne @JoinColumn(name = "id") @MapsId
    private Topic topic;

    private long views;

    //Getters and setters omitted for brevity
}
```

Another approach would be to add this relationship into the `Topic` class, and the `topic` rows would then reference the `topic_statistics` records. For the sake of demonstrating how entity association polymorphism works, the `TopicStatistics` was chosen to be the child-side.

As natural as this Domain Model may be represented in an object-oriented programming language, transposing it to a relational database is anything but straightforward. Although SQL-99 added support for type inheritance, this feature is seldom implemented. Therefore, relational database systems are relying on tuples and relational algebra for representing and manipulating data. For this reason, mapping inheritance in a relational database is one of the most obvious object-relational impedance mismatches.

Without native support from the database system, inheritance can only be emulated through table relationships. In the *Patterns of Enterprise Application Architecture* book, Martin Fowler defines three ways of mapping inheritance into a relational database:

- Single Table Inheritance[2], which uses a single database table to represent all classes in a given inheritance hierarchy.
- Class Table Inheritance[3], which maps each class to a table, and the inheritance association is resolved through table joins.
- Concrete Table Inheritance[4], where each table defines all fields that are either defined in the subclass or inherited from a superclass.

The JPA specification defines all these three inheritance mapping models through the following strategies:

- `InheritanceType.SINGLE_TABLE`
- `InheritanceType.JOINED`
- `InheritanceType.TABLE_PER_CLASS`.

JPA also covers the case when inheritance is only available in the Domain Model, without being mirrored into the database (e.g. `@MappedSuperclass`).

Whenever the data access layer implements a functionality without support from the underlying database system, care must be taken to ensure that application performance is not compromised. This chapter aims to analyze what trade-offs are required for employing inheritance as well as its impact on application performance.

[2] http://martinfowler.com/eaaCatalog/singleTableInheritance.html
[3] http://martinfowler.com/eaaCatalog/classTableInheritance.html
[4] http://martinfowler.com/eaaCatalog/concreteTableInheritance.html

11.1 Single table

The single table inheritance is the default JPA strategy, funneling a whole inheritance Domain Model hierarchy into a single database table.

To employ this strategy, the `Topic` entity class must be mapped with one of the following annotations:

- `@Inheritance` (being the default inheritance model, it is not mandatory to supply the strategy when using single table inheritance).
- `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)`.

The `Post` and the `Announcement` entities do not need any extra mapping (the Java inheritance semantics being sufficient).

Preserving the same layout as depicted in the Domain Model class diagram, the table relationships associated with this inheritance strategy look like this:

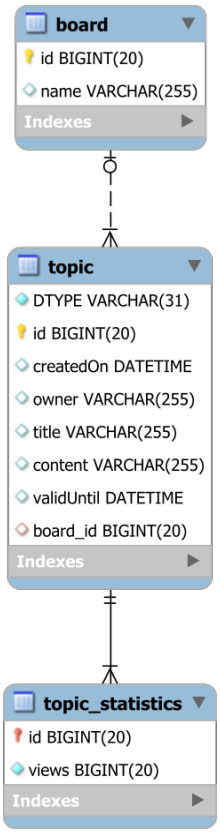

Figure 11.2: Single table

The `topic` table contains columns associated with the `Topic` base class as well as columns related to attributes from `Post` and `Announcement` entities.

In the following example, one `Post` and one `Announcement` entities are going to be persisted along with their associated `@OneToOne` `TopicStatistics` relations.

```
Post post = new Post();
post.setOwner("John Doe");
post.setTitle("Inheritance");
post.setContent("Best practices");
post.setBoard(board);

entityManager.persist(post);

Announcement announcement = new Announcement();
announcement.setOwner("John Doe");
announcement.setTitle("Release x.y.z.Final");
announcement.setValidUntil(Timestamp.valueOf(LocalDateTime.now().plusMonths(1)));
announcement.setBoard(board);

entityManager.persist(announcement);

TopicStatistics postStatistics = new TopicStatistics(post);
postStatistics.incrementViews();
entityManager.persist(postStatistics);

TopicStatistics announcementStatistics = new TopicStatistics(announcement);
announcementStatistics.incrementViews();
entityManager.persist(announcementStatistics);
```

Both the `Post` and the `Announcement` entities are saved in the `topic` table whose primary key is shared with the `topic_statistics` table.

```
INSERT INTO topic (board_id, createdOn, owner, title, content, DTYPE, id)
VALUES (1, '2016-01-17 09:22:22.11', 'John Doe', 'Inheritance',
    'Best practices', 'Post', 2)

INSERT INTO topic (board_id, createdOn, owner, title, validUntil, DTYPE, id)
VALUES (1, '2016-01-17 09:22:22.11', 'John Doe', 'Release x.y.z.Final',
    '2016-02-17 09:22:22.114', 'Announcement', 3)

INSERT INTO topic_statistics (views, id) VALUES (1, 2)

INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

One advantage of using inheritance in the Domain Model is the support for polymorphic queries. When the application developer issues a select query against the `Topic` entity:

```
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
.setParameter("boardId", 1L)
.getResultList();
```

Hibernate goes to the `topic` table, and, after fetching the result set, it maps every row to its associated subclass instance (e.g. `Post` or `Announcement`) by analyzing the discriminator column (e.g. `DTYPE`) value.

```
SELECT t.id AS id2_1_, t.board_id AS board_id8_1_, t.createdOn AS created03_1_,
    t.owner AS owner4_1_, t.title AS title5_1_, t.content AS content6_1_,
    t.validUntil AS validUnt7_1_, t.DTYPE AS DTYPE1_1_
FROM topic t
WHERE t.board_id = 1
```

Domain Model inheritance allows base class entity associations to be automatically resolved upon being retrieved. When loading a `TopicStatistics` along with its `Topic` relation:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
.setParameter("topicId", topicId)
.getSingleResult();
```

Hibernate joins the `topic_statistics` and the `topic` tables so that it can create a `TopicStatistics` entity with an actual `Post` or `Announcement` attribute object reference.

```
SELECT
    ts.id AS id1_2_0_, t.id AS id2_1_1_, ts.views AS views2_2_0_,
    t.board_id AS board_id8_1_1_, t.createdOn AS created03_1_1_,
    t.owner AS owner4_1_1_, t.title AS title5_1_1_, t.content AS content6_1_1_,
    t.validUntil AS validUnt7_1_1_, t.DTYPE AS DTYPE1_1_1_
FROM topic_statistics ts
INNER JOIN topic t ON ts.id = t.id
WHERE t.id = 2
```

Even if not practical in this particular example, `@OneToMany` associations are also possible.

The `Board` entity can map a bidirectional `@OneToMany` relationship as follows:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the aforementioned `Topic` entity query. When fetching the collection eagerly, Hibernate requires a single table join.

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
.setParameter("id", id)
.getSingleResult();
```

```
SELECT b.id AS id1_0_0_, t.id AS id2_1_1_, b.name AS name2_0_0_,
    t.board_id AS board_id8_1_1_, t.createdOn AS createdO3_1_1_,
    t.owner AS owner4_1_1_, t.title AS title5_1_1_, t.content AS content6_1_1_,
    t.validUntil AS validUnt7_1_1_, t.DTYPE AS DTYPE1_1_1_,
    t.board_id AS board_id8_1_0__, t.id AS id2_1_0__
FROM board b
INNER JOIN topic t ON b.id = t.board_id
WHERE b.id = 1
```

Performance and data integrity considerations

Since only one table is used for storing entities, both reads and writes are fast. Even when using a `@ManyToOne` or a `@OneToOne` base class association, Hibernate needs a single join between parent and child tables. The `@OneToMany` base class entity relationship is also efficient since it either generates a secondary select or a single table join.

Because all subclass attributes are collocated in a single table, `NOT NULL` constraints are not allowed for columns belonging to subclasses. Being automatically inherited by all subclasses, the base class attributes may be non-nullable. From a data integrity perspective, this limitation defeats the purpose of Consistency (guaranteed by the ACID properties).

Nevertheless, the data integrity rules can be enforced through database trigger procedures or `CHECK` constraints (a column non-nullability is accounted based on the class discriminator value). Another approach is to move the check into the data access layer. Bean Validation can validate `@NotNull` attributes at runtime. JPA also defines callback methods (e.g. `@PrePersist`, `@PreUpdate`) as well as entity listeners (e.g. `@EntityListeners`) which can throw an exception when a non-null constraint is violated.

11.1.1 Data integrity constraints

The SQL standard defines the `CHECK` constraint which can be used to apply a row-level verification for each table record that is inserted. Depending on the underlying database, the `CHECK` constraint can be enforced (e.g. Oracle, SQL Server, PostgreSQL, MySQL 8.0.16[5]) or ignored (e.g. MySQL prior to 8.0.16[6]).

For the aforementioned database tables, the `content` column must never be null if the underlying record is a `Post`, and the `validUntil` column should not be null if the database row represents an `Announcement` entity. Luckily, the default `DTYPE` column specifies the entity type associated with each particular table row.

To ensure the aforementioned data integrity rules, the following `CHECK` constraints needs to be added:

```
ALTER TABLE topic ADD CONSTRAINT post_content_check CHECK (
    CASE WHEN DTYPE = 'Post'
        THEN CASE WHEN content IS NOT NULL THEN 1 ELSE 0 END
        ELSE 1
    END = 1
)

ALTER TABLE topic ADD CONSTRAINT announcement_validUntil_check CHECK (
    CASE WHEN DTYPE = 'Announcement'
        THEN CASE WHEN validUntil IS NOT NULL THEN 1 ELSE 0 END
        ELSE 1
    END = 1
)
```

With these `CHECK` constraints in place, when trying to insert a `Post` entity without a `content`:

```
entityManager.persist(new Post());
```

PostgreSQL generates the following error message:

```
INSERT INTO topic (board_id, createdOn, owner, title, content, DTYPE, id)
VALUES ((NULL(BIGINT), '2016-07-15 13:45:16.705', NULL(VARCHAR), NULL(VARCHAR),
        NULL(VARCHAR), 'Post', 4)

-- SQL Error: 0, SQLState: 23514
-- new row for relation "topic" violates check constraint "post_content_check"
```

[5] https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html
[6] http://dev.mysql.com/doc/refman/5.7/en/create-table.html

For MySQL versions older than 8.0.16, the same outcome can be achieved with database triggers. For instance, you could add the following `BEFORE INSERT` triggers:

```
CREATE
TRIGGER post_content_check BEFORE INSERT
ON topic
FOR EACH ROW
BEGIN
   IF NEW.DTYPE = 'Post'
   THEN
       IF NEW.content IS NULL
       THEN
           signal sqlstate '45000'
           set message_text = 'Post content cannot be NULL';
       END IF;
   END IF;
END;

CREATE
TRIGGER announcement_validUntil_check BEFORE INSERT
ON topic
FOR EACH ROW
BEGIN
   IF NEW.DTYPE = 'Announcement'
   THEN
       IF NEW.validUntil IS NULL
       THEN
           signal sqlstate '45000'
           set message_text = 'Announcement validUntil cannot be NULL';
       END IF;
   END IF;
END;
```

When running the previous `Post` insert, MySQL generates the following output:

```
INSERT INTO topic (board_id, createdOn, owner, title, content, DTYPE, id)
VALUES ((NULL(BIGINT), '2016-07-15 13:50:51.989', NULL(VARCHAR), NULL(VARCHAR),
        NULL(VARCHAR), 'Post', 4)

-- SQL Error: 1644, SQLState: 45000
-- Post content cannot be NULL
```

You also need to add `BEFORE UPDATE` triggers, so that you can cover modifications for the `post` and `announcement` tables, which could compromise data integrity if the subclass-specific columns were set to `null`.

```
CREATE
TRIGGER post_content_update_check BEFORE UPDATE
ON topic
FOR EACH ROW
BEGIN
   IF NEW.DTYPE = 'Post'
   THEN
       IF NEW.content IS NULL
       THEN
           signal sqlstate '45000'
           set message_text = 'Post content cannot be NULL';
       END IF;
   END IF;
END;

CREATE
TRIGGER announcement_validUntil_update_check BEFORE UPDATE
ON topic
FOR EACH ROW
BEGIN
   IF NEW.DTYPE = 'Announcement'
   THEN
       IF NEW.validUntil IS NULL
       THEN
           signal sqlstate '45000'
           set message_text = 'Announcement validUntil cannot be NULL';
       END IF;
   END IF;
END;
```

Now, if we try to update an `Announcement` entity and set the `validUntil` column to `null`, MySQL will fail with the following error message:

```
UPDATE topic
SET board_id = 1, createdOn = '2016-07-15 13:50:51.989', OWNER = 'John Doe',
    title = 'John Doe, Release x.y.z.Final', validUntil = NULL(TIMESTAMP)
WHERE id = 3

-- SQL Error: 1644, SQLState: 45000
-- Announcement validUntil cannot be NULL
```

Although a little bit verbose, the `CHECK` and `TRIGGER` constraints are very useful to ensure data integrity when using single table inheritance.

11.2 Join table

The join table inheritance resembles the Domain Model class diagram since each class is mapped to an individual table. The subclass tables have a foreign key column referencing the base class table primary key.

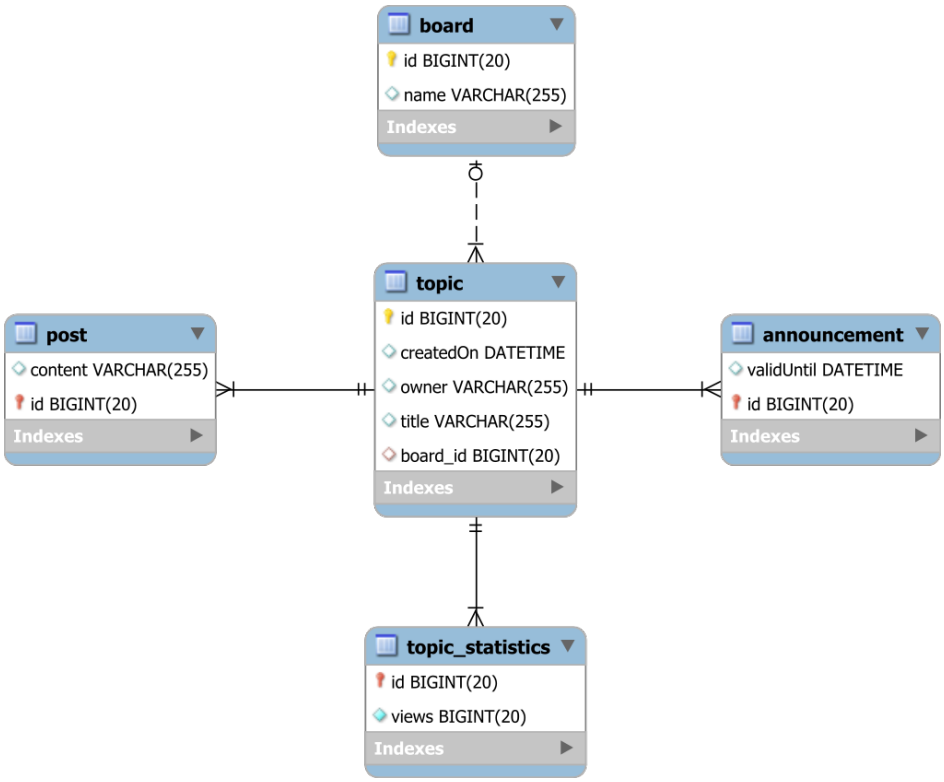

Figure 11.3: Join table

To use this inheritance strategy, the `Topic` entity must be annotated with:

```
@Inheritance(strategy = InheritanceType.JOINED)
```

The `Post` and the `Announcement` entities can use a `@PrimaryKeyJoinColumn` mapping to define the base class foreign key column.

By default, the subclass table primary key column is used as a foreign key as well.

When persisting the same entities defined in the single table section, Hibernate generates the following SQL statements:

```
INSERT INTO topic (board_id, createdOn, owner, title, id)
VALUES (1, '2016-01-17 09:27:10.694', 'John Doe', 'Inheritance', 2)

INSERT INTO post (content, id) VALUES ('Best practices', 2)

INSERT INTO topic (board_id, createdOn, owner, title, id)
VALUES (1, '2016-01-17 09:27:10.694', 'John Doe', 'Release x.y.z.Final', 3)

INSERT INTO announcement (validUntil, id) VALUES ('2016-02-17 09:27:10.698', 3)

INSERT INTO topic_statistics (views, id) VALUES (1, 2)

INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

The base class information goes into the `topic` table while the subclass content goes in the `post` or the `announcement` tables. When fetching all `Topic` entities associated with a specific `Board`:

```
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
.setParameter("boardId", 1L)
.getResultList();
```

Hibernate must join the base class with each individual subclass table.

```
SELECT
    t.id AS id1_3_,
    t.board_id AS board_id5_3_,
    t.createdOn AS createdO2_3_,
    t.owner AS owner3_3_,
    t.title AS title4_3_,
    t1_.content AS content1_2_,
    t2_.validUntil AS validUnt1_0_,
    CASE WHEN t1_.id IS NOT NULL THEN 1
        WHEN t2_.id IS NOT NULL THEN 2
        WHEN t.id IS NOT NULL THEN 0
    END AS clazz_
FROM topic t
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE t.board_id = 1
```

When loading a `TopicStatistics` entity along with its `Topic` association:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
.setParameter("topicId", topicId)
.getSingleResult();
```

Hibernate must join four tables to construct the desired result set:

```
SELECT
    ts.id AS id1_4_0_,
    t.id AS id1_3_1_,
    ts.views AS views2_4_0_,
    t.board_id AS board_id5_3_1_,
    t.createdOn AS createdO2_3_1_,
    t.owner AS owner3_3_1_,
    t.title AS title4_3_1_,
    t1_.content AS content1_2_1_,
    t2_.validUntil AS validUnt1_0_1_,
    CASE WHEN t1_.id IS NOT NULL THEN 1
        WHEN t2_.id IS NOT NULL THEN 2
        WHEN t.id IS NOT NULL THEN 0
    END AS clazz_1_
FROM topic_statistics ts
INNER JOIN topic t ON ts.id = t.id
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE t.id = 2
```

Considering that the `Board` entity defines a `@OneToMany` `Topic` association:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the previous `Topic` entity query.

When fetching the collection eagerly:

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
.setParameter("id", id)
.getSingleResult();
```

Hibernate requires three joins to fetch all topic-related information.

```
SELECT
    b.id AS id1_1_0_, t.id AS id1_3_1_, b.name AS name2_1_0_,
    t.board_id AS board_id5_3_1_, t.createdOn AS createdO2_3_1_,
    t.owner AS owner3_3_1_, t.title AS title4_3_1_,
    t1_.content AS content1_2_1_, t2_.validUntil AS validUnt1_0_1_,
    CASE WHEN t1_.id IS NOT NULL THEN 1
         WHEN t2_.id IS NOT NULL THEN 2
         WHEN t.id IS NOT NULL THEN 0
    END AS clazz_1_,
    t.board_id AS board_id5_3_0__, t.id AS id1_3_0__
FROM board b
INNER JOIN topic t ON b.id = t.board_id
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE b.id = 1
```

Performance considerations

Unlike single table inheritance, the joined table strategy allows nullable subclass attribute columns.

When writing data, Hibernate requires two insert statements for each subclass entity, so there's a performance impact compared to single table inheritance. The index memory footprint also increases because instead of a single table primary key, the database must index the base class and all subclasses primary keys.

When reading data, polymorphic queries require joining the base class with all subclass tables, so, if there are `n` subclasses, Hibernate will need N+1 joins. The more joins, the more difficult it is for the database to calculate the most efficient execution plan.

11.3 Table-per-class

The table-per-class inheritance model has a table layout similar to the joined table strategy, but, instead of storing base class columns in the `topic` table, each subclass table also stores columns from the `topic` table. There is no foreign key between the `topic` and the `post` or `announcement` subclass tables, and there is no foreign key in the `topic_statistics` table either.

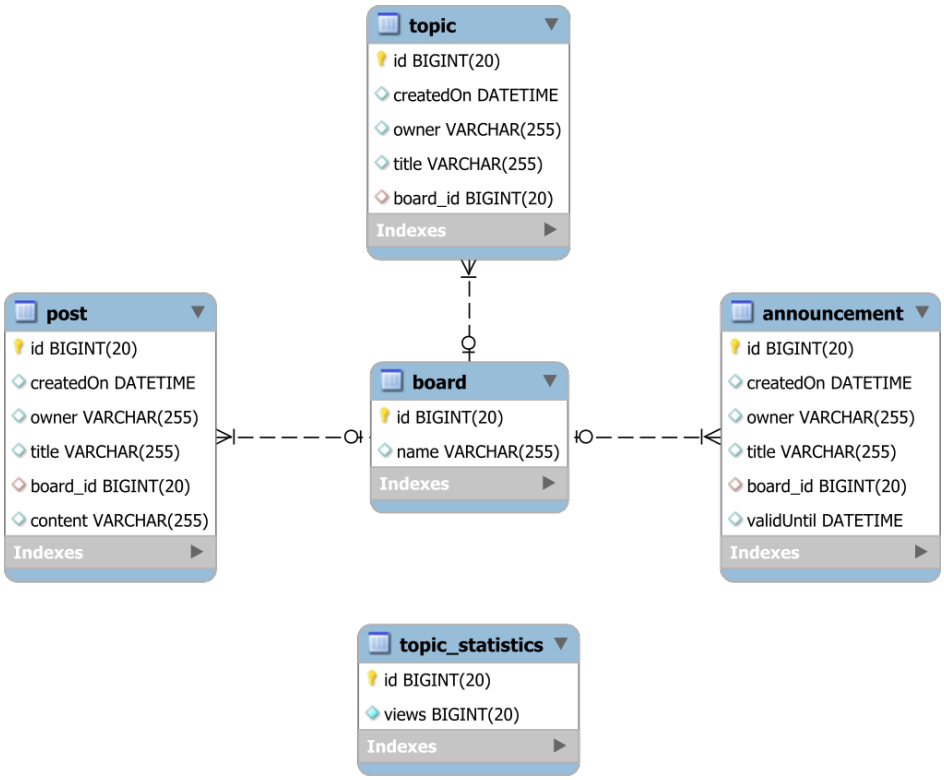

Figure 11.4: Table-per-class

To use this inheritance strategy, the `Topic` must be annotated with `@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)`.

Inserting the `Post` and the `Announcement` entities defined in the single table inheritance section generates the following SQL statements:

```
INSERT INTO post (board_id, createdOn, owner, title, content, id)
VALUES (1, '2016-01-17 09:31:12.018', 'John Doe', 'Inheritance',
        'Best practices', 2)

INSERT INTO announcement (board_id, createdOn, owner, title, validUntil, id)
VALUES (1, '2016-01-17 09:31:12.018', 'John Doe', 'Release x.y.z.Final',
        '2016-02-17 09:31:12.023', 3)
```

```sql
INSERT INTO topic_statistics (views, id) VALUES (1, 2)

INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

Unlike the joined table inheritance, each persisted subclass entity requires a single insert statement.

When fetching all `Topic` entities associated with a specific `Board`:

```java
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
.setParameter("boardId", 1L)
.getResultList();
```

Hibernate uses `UNION ALL` to fetch rows from the base class and every subclass table in this particular inheritance tree.

```sql
SELECT
    t.id AS id1_3_,
    t.board_id AS board_id5_3_,
    t.createdOn AS createdO2_3_,
    t.owner AS owner3_3_,
    t.title AS title4_3_,
    t.content AS content1_2_,
    t.validUntil AS validUnt1_0_,
    t.clazz_ AS clazz_
FROM (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id, content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content, validUntil, 2 AS clazz_
    FROM announcement ) t
WHERE  t.board_id = 1
```

When loading a `TopicStatistics` while also fetching its associated `Topic` relation, Hibernate must use `UNION ALL` for the inheritance tables to construct the desired result set:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
.setParameter("topicId", topicId)
.getSingleResult();
```

```
SELECT
    ts.id AS id1_4_0_, t.id AS id1_3_1_, ts.views AS views2_4_0_,
    t.board_id AS board_id5_3_1_, t.createdOn AS createdO2_3_1_,
    t.owner AS owner3_3_1_, t.title AS title4_3_1_, t.content AS content1_2_1_,
    t.validUntil AS validUnt1_0_1_, t.clazz_ AS clazz_1_
FROM topic_statistics ts
INNER JOIN (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        validUntil, 2 AS clazz_
    FROM announcement
) t ON ts.id = t.id
WHERE t.id = 2
```

The identity generator is not allowed with this strategy because rows belonging to different subclasses would share the same identifier, therefore conflicting in polymorphic `@ManyToOne` or `@OneToOne` associations.

Considering that the `Board` entity defines a `@OneToMany` `Topic` association:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the previous `Topic` entity query.

When fetching the `topics` collection eagerly:

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
.setParameter("id", id)
.getSingleResult();
```

Hibernate requires a join with the result of unifying all three topic-related tables.

```
SELECT
    b.id AS id1_1_0_, t1.id AS id1_3_1_, b.name AS name2_1_0_,
    t1.board_id AS board_id5_3_1_, t1.createdOn AS createdO2_3_1_,
    t1.owner AS owner3_3_1_, t1.title AS title4_3_1_,
    t1.content AS content1_2_1_, t1.validUntil AS validUnt1_0_1_,
    t1.clazz_ AS clazz_1_, t1.board_id AS board_id5_3_0__, t1.id AS id1_3_0__
FROM board b
INNER JOIN (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        validUntil, 2 AS clazz_
    FROM announcement
) t1 ON b.id = t1.board_id
WHERE b.id = 1
```

Performance considerations

While write operations are faster than for the joined table strategy, the read operations are only efficient when querying against the actual subclass entities. Polymorphic queries can have a considerable performance impact because Hibernate must select all subclass tables and use `UNION ALL` to build the whole inheritance tree result set. As a rule of thumb, the more subclass tables, the less efficient the polymorphic queries get.

11.4 Mapped superclass

If the `Topic` class is not required to be a stand-alone entity, it will be more practical to leave inheritance out of the database. This way, the `Topic` can be made `abstract` and marked with the `@MappedSuperclass` annotation so that JPA can acknowledge the inheritance model on the entity-side only.

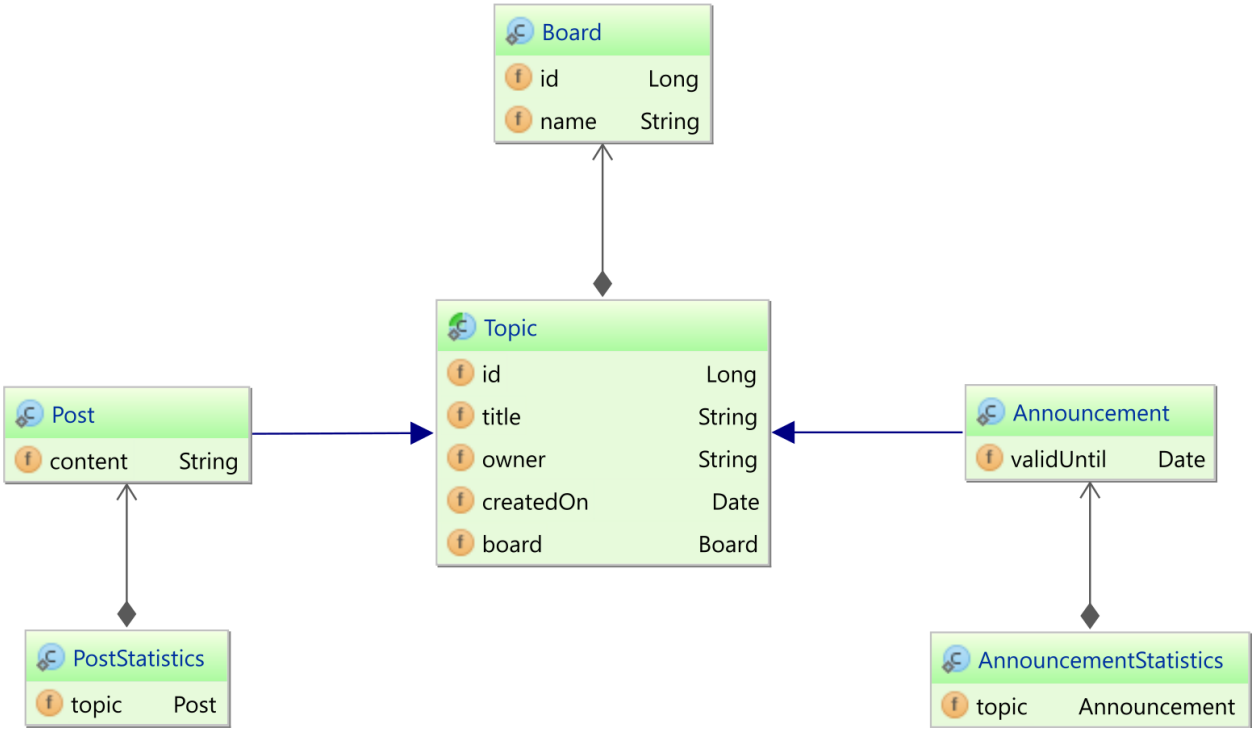

Figure 11.5: Mapped superclass class diagram

Having a single `TopicStatistics` entity with a `@OneToOne Topic` association is no longer possible because the `Topic` class is not an entity. This way, each `Topic` subclass must define its own statistics associations (e.g. `PostStatistics`, `AnnouncementStatistics`).

The `PostStatistics` and the `AnnouncementStatistics` entities looks as follows:

```
@Entity @Table(name = "post_statistics")
public class PostStatistics
    extends TopicStatistics<Post> {

    @OneToOne
    @JoinColumn(name = "id")
    @MapsId
    private Post topic;

    private PostStatistics() {}

    public PostStatistics(Post topic) {
        this.topic = topic;
    }

    @Override
    public Post getTopic() {
        return topic;
    }
}

@Entity @Table(name = "announcement_statistics")
public class AnnouncementStatistics
    extends TopicStatistics<Announcement> {

    @OneToOne
    @JoinColumn(name = "id")
    @MapsId
    private Announcement topic;

    private AnnouncementStatistics() {}

    public AnnouncementStatistics(Announcement topic) {
        this.topic = topic;
    }

    @Override
    public Announcement getTopic() {
        return topic;
    }
}
```